OPERATORS

What is an Operator?

- In JavaScript, an operator is a special symbol used to perform operations on operands (values and variables).
- For example,
- 2 + 3; // 5
- Here + is an operator that performs addition, and 2 and 3 are operands.

Operator Types

1. Assignment Operators
2. Arithmetic Operators
3. Comparison Operators
4. Logical Operators
5. Bitwise Operators
6. String Operators
7. Other Operators

1 Assignment Operators

- Assignment operators are used to assign values to variables.
- For example,

```
const x = 5;
```

- Here, the = operator is used to assign value 5 to variable x.

Shorthands

- += Addition assignment a += 5; // a = a + 5
- -= Subtraction Assignment a -= 2; // a = a - 2
- *= Multiplication Assignment a *= 3; // a = a * 3
- /= Division Assignment a /= 2; // a = a / 2
- %= Remainder Assignment a %= 2; // a = a % 2
- **= Exponentiation Assignment a **= 2; // a = a**2

2 Arithmetic Operators

- Arithmetic operators are used to perform arithmetic calculations.
- For example,

  ```
  const number = 3 + 5; // 8
  ```

- Here, the + operator is used to add two operands.

Operators Example

+	Addition	x + y	
-	Subtraction	x - y	
*	Multiplication		x * y
/	Division	x / y	
%	Remainder	x % y	
++	Increment (increments by 1)		++x or x++
--	Decrement (decrements by 1)	--x or x--	
**	Exponentiation (Power)		x ** y

3 Comparison Operators (Relational operator)

- Comparison operators compare two values and return a Boolean value, either true or false. For example,
- const a = 3, b = 2;
- console.log (a > b); // true
- Here, the comparison operator > is used to compare whether a is greater than b.

- == Equal to: returns true if the operands are equal x==y
- != Not equal to: returns true if the operands are not equal x != y
- === Strict equal to: true if the operands are equal and of the same type x === y
- !== Strict not equal to: true if the operands are equal but of different type x !== y

  or not equal at all

- >Greater than: true if left operand is greater than the right operand x > y
- >= Greater than or equal to: true if left operand is greater than or equal to the right operand x >= y
- <Less than: true if the left operand is less than the right operand x < y
- <= Less than or equal to: true if the left operand is less than or equal to the right operand x <= y

4 Logical Operators

- Logical operators perform logical operations and return a Boolean value, either true or false. For example,
- const x = 5, y = 3;
- (x < 6) && (y < 5); // true
- Here, && is the logical operator AND. Since both x < 6 and y < 5 are true, the result is true.

- && **Logical AND**: true if both the operands are true, else returns false x && y
- || **Logical OR:** true if either of the operands is true; returns false x || y

  if both are false
- ! **Logical NOT**: true if the operand is false and vice-versa. !x

5 Bitwise Operators

- & Bitwise AND
- | Bitwise OR
- ^ Bitwise XOR
- ~ Bitwise NOT
- << Left shift
- >> Sign-propagating right shift
- >>> Zero-fill right shift

6 Other JavaScript Operators

- , Evaluates multiple operands and returns the value of the last operand.

  E.g. let a = (1, 3 , 4); // 4

- **?:** Returns value based on the condition

  e.g.: (5 > 3) ? 'success' : 'error'; // "success"

- **delete** deletes an object's property, or an element of an array
  e.g.: delete x

- **typeof** returns a string indicating the data type

  e.g.: typeof 3; // "number"

- **void** discards the expression's return value

  e.g.: void(x)

- **in** returns true if the specified property is in the object

  e.g.: prop in object

- **instanceof** returns true if the specified object is of the specified object type

  e.g.: object instanceof object_type